Distributed P2P File Sharing System

Technical Report

Hritik Ranjan

1 Overview

The system is a hybrid peer-to-peer file sharing service. Two *trackers* keep metadata (users, groups, file and piece hashes, who is seeding) and replicate it to each other. *Clients* hash the files they share, serve pieces to other clients and download from several peers in parallel. File bytes never pass through, or are stored on, a tracker. Files are split into 512 KB pieces; every piece and the whole file carry a SHA-1 hash and every received piece is verified before it is written.

2 Implementation approach

Code is split by responsibility (table below). The tracker is a thread-per-connection server with one coarse mutex around the in-memory state; the client has a command-line thread, a peer-server thread per incoming connection, and a worker pool per download.

Module	Responsibility
`tracker.cpp`	accept loop, per-connection thread, numeric status codes
`user_manager.cpp`	users, sessions, groups, command dispatch
`file_manager.cpp`	file metadata: upload, list, `file_info`, `add_seeder`, `stop_share`
`store.cpp`	versioned, replicated state (cells, hybrid logical clock, merge)
`sync_trackers.cpp`	tracker-to-tracker transport, heartbeat, reconnect, state exchange
`client.cpp`	CLI, tracker connection, failover, local hashing for uploads
`peer_server.cpp`	answers `GET_PIECE`; authorisation, bounds checks
`downloader.cpp`	worker pool, peer selection, retry, `pwrite`, completion
`net_util.h`	`sendAll`/`recvAll`/`LineReader`, framing, SHA-1 via OpenSSL EVP

3 Protocol design and rationale

All communication uses TCP with text headers; payload bytes are binary.

Client → tracker. A request is one line. Commands that carry file names use TAB as the field separator, so names may contain spaces. A reply is a frame `<length> <status>\n <payload>`. Length-prefixing removes any guessing about where a reply ends (an earlier design relied on a read timeout and truncated long listings); the numeric status (200, 400, 401, 403, 404, 409; 416/500 in the peer protocol) lets programs react without parsing the text shown to the user.

Client ↔ client. `GET_PIECE` over a persistent connection, answered by `PIECE_DATA <n>` followed by exactly n bytes, or `ERROR <code> <reason>`. A persistent connection means a 1 GB download needs one connection per (worker, peer), not 2048 connections. The peer server asks its tracker whether the requester's (user, token) is a logged-in member of the group and checks the piece index against the real file size.

Partial transmissions. Every read and write goes through helpers that loop until the whole length is transferred; `LineReader` buffers per connection so several commands in one segment, or one command split over several segments, are both handled. `SIGPIPE` is ignored.

4 Synchronization algorithm

Problem. If both trackers stay up but cannot reach each other, both keep serving clients. Copying one side's snapshot over the other on reconnect loses the other side's changes and revives deleted records.

Design. All state is a set of *cells*: `key -> (stamp, present, value)`. Kinds: user, login, group, member, join request, file, seeder, shared-file. A deletion is a cell with `present=false` and its own stamp (a *tombstone*). The `stamp` is a hybrid logical clock (wall ms, counter, tracker id) compared lexicographically [2]: the clock never runs backwards, a change made after observing a remote change always gets a larger stamp even with clock skew, and the tracker id breaks ties deterministically. Merging is

$$\text{merge}(c_1, c_2) = \text{the cell with the larger stamp},$$

applied per key. It is commutative, associative and idempotent, so the cells form a state-based last-writer-wins element set [4]; any delivery order or duplication converges.

Protocol. A local change is stamped, applied to the in-memory views (the `users`/`groups` maps that command handlers read) and queued; after the data mutex is released it is sent as one `CELL` line. Whenever a tracker's outgoing connection is (re)established it sends *every* cell including tombstones and asks the peer to do the same. The heartbeat (3 s) is also the reconnect timer. This single mechanism provides (i) restart recovery (an empty tracker is refilled), and (ii) partition healing.

Why it is correct for the tested cases. During a cut, tracker 1 deleting a file stamps a tombstone t_1; tracker 2's older copy of the file has stamp $s < t_1$, so after exchange the tombstone wins on both sides. Independent additions (a group created on each side) touch different keys and both survive. A cell that arrives before its parent (a seeder before its file) is stored, and attached when the parent arrives, by a prefix scan of the ordered map.

Deadlock avoidance. Lock order is sync mutex → data mutex; the replication queue is flushed only after the data mutex is released; a snapshot request is answered on its own thread, so two trackers asking each other for state cannot block in `send()` at the same time.

Sessions. Login issues a random 128-bit token (replicated). After a tracker dies, the client reconnects to the other and calls `restore_session user token`. A dropped client connection is processed exactly like `logout` and replicated. A tracker `quit` does not log users out.

5 Piece selection strategy

Every seeder holds the complete file, so there is no piece rarity to exploit; rarest-first would degenerate to arbitrary order. The engine therefore uses *round-robin load balancing*: piece i is first requested from peers[$(i + \text{attempt}) \bmod n$]. Each live seeder serves an equal share and no piece is requested twice. A fixed pool of workers (default 8) pulls piece indices from a shared queue (mutex and condition variable). A piece that fails (connection error, wrong length, hash mismatch) is re-queued with that peer excluded and an attempt counter, up to 6 attempts; three consecutive failures mark a peer dead for the download. Verified pieces are written with `pwrite()` at `index`

`* 512 KB`, so no file lock is needed. Completion is decided by counters (`completed + failed == total`), after which the whole file is verified, the client starts serving it and registers with the tracker (`add_seeder`). Several downloads may run at once.

6 Testing and results

`scripts/integration_test.py` starts real trackers and clients and checks 96 properties: file sizes from 0 bytes to 80 MB and several file types; byte-exact downloads; load balance; retry when a seeder is killed mid-transfer; failure with no partial file when none is left; tracker crash, failover, restart catch-up; a *network partition between two live trackers* (sync channel cut by a TCP proxy) with additions and deletions on both sides, then heal; glued and split protocol messages (including a 200 KB sync message in 1.5 KB pieces); authorisation, bounds, status codes. The suite also passes under ThreadSanitizer, AddressSanitizer+UBSan, and the macOS `leaks` tool reports 0 leaks in every live process afterwards.

Benchmark (`scripts/benchmark.py`; random 1 GB file, Apple M4, loopback, 8 workers; median of 3 runs after a discarded warm-up).

	Unthrottled		Seeder uplink emulated at 100 MB/s		
Seeders	Time	vs 1	Time	MB/s	vs 1
1	0.175 s	1.00×	10.248 s	100	1.00×
2	0.181 s	0.97×	5.128 s	200	2.00×
4	0.185 s	0.95×	2.572 s	398	3.98×

Unthrottled, the downloader's CPU (SHA-1 of every piece, memory copies) is the bottleneck, so extra seeders do not help. The right-hand columns come from a token bucket in the peer server (`P2P_UPLOAD_KBPS`); they show that the engine keeps every seeder saturated and splits work evenly, not a real network measurement.

7 Challenges and solutions

Knowing when a download is finished.
Inferring failure from "no active threads" fired before workers had started. Solution: count pieces completed/failed under one mutex.

Partial reads and writes.
A single `read()` is not a message. Solution: `sendAll`/`recvAll`, buffered `LineReader`, length-prefixed replies.

Failover without split state.
A dropped connection and an explicit logout used to differ, so the two trackers disagreed. Solution: one code path for both, replicated; session token for `restore_session`.

Partitions between live trackers.
See section 4; solved with versioned cells and tombstones, and tested with a TCP proxy that cuts the sync channel.

Lock ordering and network I/O.
Sending while holding the data mutex risked stalls and deadlocks. Solution: queue messages under the lock, flush after release; one documented lock order.

Automatic seeding.
The tracker listing a downloader as seeder is useless unless its peer server can serve the file. Solution: downloads register the file locally first, then tell the tracker; failed uploads register nothing.

Memory limits.
A 512 KB array on a thread stack can overflow secondary-thread stacks. Solution: heap buffers.

Honest performance numbers.
Loopback hides what multi-source download is for. Solution: report the unthrottled result as measured and the capped one explicitly as emulation.

8 Limitations

Two trackers only; concurrent conflicting edits to one record during a partition resolve by latest stamp; tombstones are not garbage-collected; no TLS (passwords are hashed at rest and in replication but sent in clear between client and tracker); a client's list of files it serves is in memory (re-attach with `become_seeder` after a restart); no per-peer piece bitmaps, so only complete files are seeded; developed and tested on macOS.

References

[1] B. Cohen, *The BitTorrent Protocol Specification* (BEP 3), https://www.bittorrent.org/beps/bep_0003.html – background on trackers, pieces and per-piece hashing.

[2] S. Kulkarni, M. Demirbas, D. Madappa, B. Avva, M. Leone, *Logical Physical Clocks and Consistent Snapshots in Globally Distributed Databases*, 2014 – hybrid logical clock.

[3] L. Lamport, *Time, Clocks, and the Ordering of Events in a Distributed System*, CACM 21(7), 1978.

[4] M. Shapiro, N. Preguiça, C. Baquero, M. Zawirski, *A comprehensive study of Convergent and Commutative Replicated Data Types*, INRIA RR-7506, 2011 – LWW-element-set.

[5] D. Eastlake, P. Jones, *US Secure Hash Algorithm 1 (SHA1)*, RFC 3174.

[6] POSIX.1-2017 (`socket`, `poll`, `pread`/`pwrite`, `pthread`) and the OpenSSL `EVP_Digest*` manual pages.